

Service to Incident

Table of Contents

1. Purpose	1
2. The four gates	1
3. Tag flow: Kubernetes to ServiceNow	2
4. How the CI relates to the Service Instance	2
5. Status on this implementation	3

1. Purpose

This is a companion to [Log to Incident](#), scoped to one question: **when a Dynatrace `SERVICE-*` entity misbehaves (for example, degraded response time), what has to be true for that to show up in ServiceNow, and which ServiceNow CI does it land on?**

Log to Incident's alert-to-CI binding was designed and verified against infrastructure alerts (host CPU, log signatures). Extending that same pipeline to native Dynatrace service-level problems surfaced four separate gates that all have to be independently satisfied. None of them is optional, and none of them implies the others. This document exists so the next person adding a service-level alert doesn't have to rediscover them.

2. The four gates

Gate	What it controls
1. Management zone membership	A <code>SERVICE-</code> entity only joins a management zone if a rule explicitly targets <code>entityType: SERVICE</code> (or <code>PROCESS_GROUP</code> with <code>pgToServicePropagation</code>). A <code>HOST</code> rule with <code>hostToPGPropagation</code> stops at the process group – <code>pgToServicePropagation</code> is a <code>*different</code> field whose schema precondition requires <code>entityType: PROCESS_GROUP</code> , so setting it on a <code>HOST</code> rule is silently accepted and silently ignored. Without a service-scoped rule, every <code>SERVICE-*</code> entity on Spark hosts has an empty management zone, no matter how obviously it belongs there.
2. Alerting profile scope	The profile that forwards problems to the ServiceNow webhook (<code>spark-alerting-profile.json.j2</code>) filters by management zone. Gate 1 is a hard prerequisite for Gate 2: a problem on an entity outside the zone is invisible to the profile regardless of severity.
3. Alert-to-CI resolution	Once a problem does reach ServiceNow as an <code>em_event</code> , <code>ResolveApplicationService.si.js</code> resolves <code>em_alert.cmdb_ci</code> directly from the event payload (host, pod, or process-group identity). It never consults the CSDM Service Instance association described in How the CI relates to the Service Instance . A <code>SERVICE-*</code> -scoped problem lands on the same host/pod CI an infrastructure alert would — there is no code path that binds an alert to a Service Instance CI.

Gate	What it controls
4. Detection sensitivity	Dynatrace's default response-time detection is adaptive-baseline (auto mode): it compares live response time against a rolling historical baseline. A newly-discovered service has no baseline, so a short load burst produces no anomaly even though the detector is correctly configured. Either let real traffic accumulate first, or switch to fixed thresholds for a deterministic test.

3. Tag flow: Kubernetes to ServiceNow

One Kubernetes pod label is the single source of truth for identity across every downstream system:

```
servicenow.com/service-instance: Spark-Worker # set once, in the pod template
```

Two independent paths carry it forward:

- **Through Dynatrace's entity model.** OneAgent's Kubernetes metadata enrichment stamps the label onto the pod's `PROCESS_GROUP_INSTANCE` as an Environment tag, and it propagates to `SERVICE-*` entities running inside that process (verified directly against a live entity: the tag is present with `source: Environment tags`).
- **Through the metrics pipeline.** The same label survives Prometheus's pod-label relabeling, becomes an OTel resource attribute, and lands as the `servicenow_com_service_instance` dimension on every Dynatrace metric data point scraped from that pod — independent of OneAgent, and useful even where OneAgent tagging alone wouldn't reach (custom metrics, not entities).

Neither path requires ServiceNow to be involved yet. ServiceNow only enters at Gate 4, described next.

4. How the CI relates to the Service Instance

These are two different things that happen to be linked by the same tag, not one derived from the other:

- The **CI** (`cmdb_ci_kubernetes_pod`, or the underlying host CI) is what an alert or incident actually binds to (`em_alert.cmdb_ci`, Gate 3). It represents the running infrastructure.
- The **Service Instance** (`cmdb_ci_service_by_tags`, e.g. "Spark Worker") is a CSDM object representing the logical service. ServiceNow's tag-based Service Mapping (`sm_populate_tag_list.js`) reads the same `servicenow.com/service-instance` tag off Dynatrace-discovered CIs and writes a `svc_ci_assoc` row associating the CI with the Service Instance. This determines business-service ownership and reporting — **not** which CI an alert lands on.

An alert's CI and its host's Service Instance membership are consistent in practice (they trace back to the same tag) but arrive via independent mechanisms. Changing one does not change the other; a bug in tag-based Service Mapping would not affect Gate 3's alert binding, and vice versa.

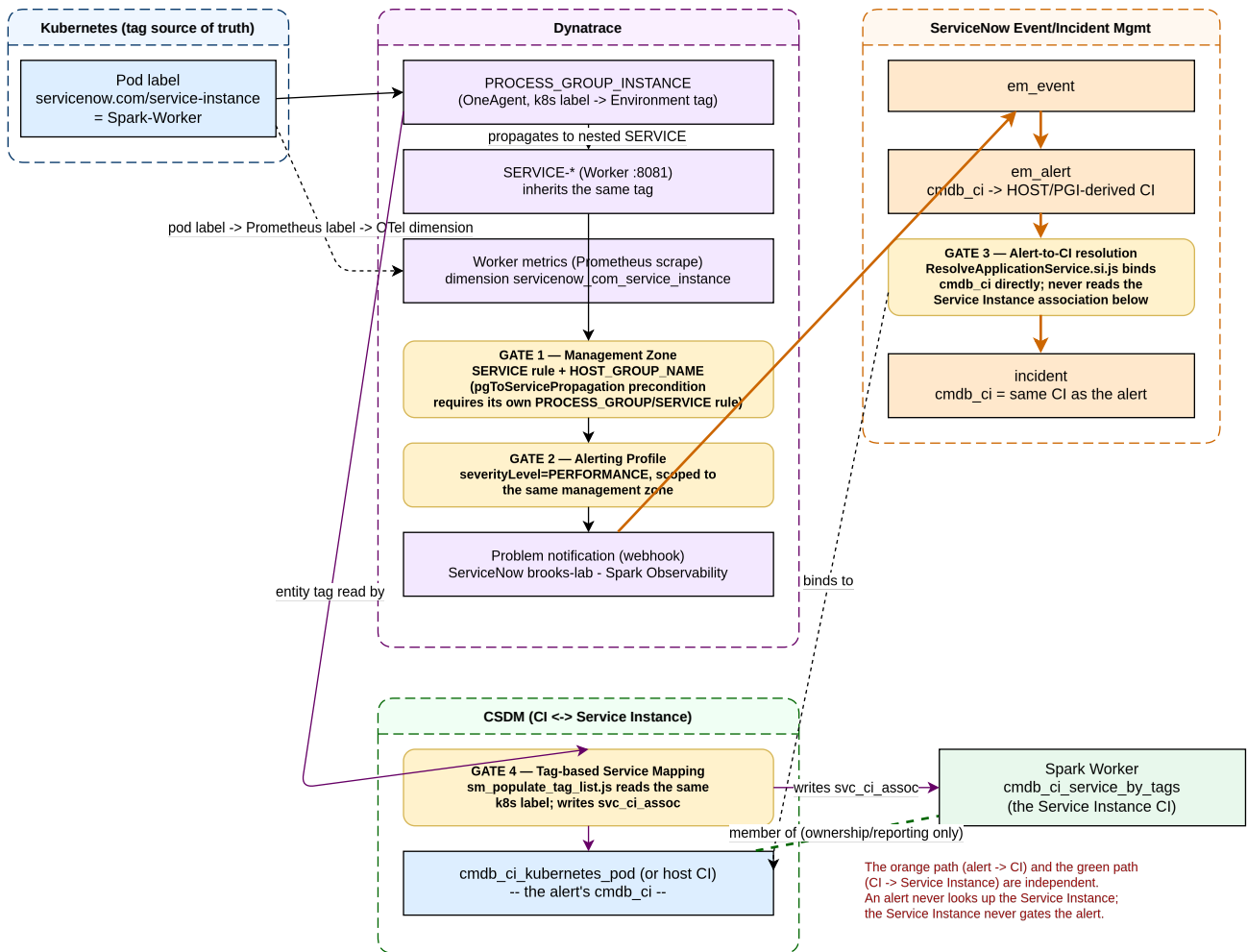


Figure 1. Tag flow and the four gates

5. Status on this implementation

As of this writing, Gates 1 and 2 are implemented and verified against the live Dynatrace tenant (management-zone rule + alerting profile scope). Gate 3 reflects existing, unchanged behavior from Log to Incident — no code path binds a Service Instance to an alert. Gate 4's tag propagation is verified for Workers via both the entity-tag and metric-dimension paths. Gate 4 has not yet produced a live, end-to-end response-time alert in ServiceNow; see the companion test results for current status.